

项目三层架构与数据流转（简洁版）

一、三层架构基本概念

项目采用“职责分离”的三层架构设计，各层专注于单一功能，协同实现数据处理与界面展示：

层级	核心职责
Models 层	定义数据的“结构与格式”（如任务、标签的数据字段），不涉及业务逻辑或界面；
Services 层	处理数据的“获取、加工、存储”（如从服务器拉取数据、本地缓存、第三方同步），是业务逻辑核心；
Views 层	展示数据给用户，接收用户操作（如点击、输入），不直接处理业务逻辑；

二、核心数据流转过程

通过“查看任务列表”“完成任务”两个典型场景，展示数据在三层间的流转逻辑：

场景 1：查看任务列表

用户进入任务页面，加载并显示当天任务，流程如下：

1. Views 层发起请求（文件：task.swift）

页面初始化时调用 `loadTasks()`，先尝试加载本地缓存，再从 Services 层获取最新数据，最后更新界面：

```
private func loadTasks() async {  
    // 1.（可选）先加载本地缓存数据，提升体验  
    // 2. 调用Services层方法，获取最新任务  
    let latestTasks = try await taskService.getTasksByDate(currentDate)  
    // 3. 更新界面状态变量，触发UI自动刷新
```

```
self.tasks = latestTasks
}
```

2. Services 层处理请求（文件：TaskService.swift）

接收 Views 层请求，发送网络请求获取数据，解析为 Models 层定义的格式，返回给 Views 层：

```
func getTasksByDate(_ date: Date) async throws -> [TodoTask] {
    // 1. 格式化日期（适配接口要求）
    // 2. 发送网络请求，获取服务器数据
    let response = try await networkService.post(
        "/api/todo_task/tasks/date",
        body: ["date": formattedDate],
        decoder: JSONDecoder()
    )
    // 3. 返回解析后的任务列表（Models层格式）
    return response.data ?? []
}
```

3. Models 层定义数据格式（文件：TodoTask.swift）

明确任务的数据结构，确保 Services 层解析、Views 层展示的数据格式统一：

```
struct TodoTask: Codable {
    let id: Int          // 任务唯一ID
    let title: String     // 任务标题
    let description: String? // 任务描述（可选）
    let dueDate: Date     // 截止日期
    let isCompleted: Bool // 是否完成
    // 其他属性...
}
```

场景 2：完成任务

用户点击“完成”按钮，标记任务为完成，流程如下：

1. Views 层调用服务（文件：task.swift）

接收用户操作，调用 Services 层“完成任务”方法，完成后刷新界面：

```
// 用户点击“完成”按钮触发
Task {
    try await taskService.completeTasks(taskIds: [task.id])
    await refreshTasks() // 刷新任务列表，更新UI
}
```

2. Services 层处理业务逻辑（文件：TaskService.swift）

执行“完成任务”的核心逻辑（通知服务器、同步第三方服务）：

```
func completeTasks(taskIds: [Int]) async throws {
    // 1. 发送网络请求，通知服务器标记任务为完成
    let response = try await networkService.post(
        "/api/todo_task/tasks/complete",
        body: ["task_ids": taskIds]
    )
    // 2. （可选）同步任务状态到第三方服务（如Google日历）
    if calendarService.isAuthorized {
        await calendarService.syncCompletedTasks(taskIds: taskIds)
    }
    // 3. 处理响应异常（如服务器返回失败）
    guard response.success else {
        throw TaskError.serverError(message: response.message ?? "操作失败")
    }
}
```

三、总结

三层架构类比“工厂流水线”，数据流转逻辑清晰：

- **Models 层** = 产品设计图（定义数据格式）；
- **Services 层** = 生产车间（处理数据获取、业务逻辑）；
- **Views 层** = 产品展示柜（展示数据、接收用户操作）。

数据流转方向：

服务器数据 → Services 层处理 → Models 层格式化 → Views 层展示；

用户操作 → Views 层触发 → Services 层执行 → 服务器同步。

该架构确保代码职责清晰、易维护，便于后续功能扩展。

（注：文档部分内容可能由 AI 生成）